

ΟΜΑΔΑ Α. ΓΕΝΙΚΕΣ ΕΡΩΤΗΣΕΙΣ

A1. Ρόλος Λειτουργικού Συστήματος

Το Λειτουργικό Σύστημα είναι το βασικό λογισμικό που διαχειρίζεται τους πόρους του υπολογιστή (CPU, μνήμη, δίσκους, συσκευές) και παρέχει υπηρεσίες στα προγράμματα και στο χρήστη. Χωρίς ΛΣ δεν θα μπορούσαν να φορτωθούν/εκτελεστούν προγράμματα, να οργανωθούν αρχεία σε δίσκους ή να γίνει επικοινωνία με τις συσκευές. Ουσιαστικά ο χρήστης θα έπρεπε να γράφει κώδικα χαμηλού επιπέδου για κάθε στοιχειώδη λειτουργία.

A2. Κύρια μέρη ΛΣ – Πυρήνας

Κύρια μέρη ενός ΛΣ: πυρήνας (kernel), διαχείριση μνήμης, σύστημα αρχείων, διαχείριση διεργασιών, διαχείριση συσκευών I/O, διεπαφή χρήστη (CLI/GUI), βοηθητικά εργαλεία. Ο πυρήνας είναι το κεντρικό τμήμα που εκτελείται σε προνομιακό επίπεδο και ελέγχει άμεσα το υλικό. Διαχειρίζεται διεργασίες, μνήμη, συσκευές και μεταβάσεις κατάστασης (interrupts, system calls).

A3. Διαδοχική και Άμεση οργάνωση αρχείων

Στη διαδοχική (sequential) οργάνωση, οι εγγραφές είναι αποθηκευμένες η μία μετά την άλλη και προσπελάζονται με σειρά, από την αρχή προς το τέλος. Είναι κατάλληλη για σειριακή επεξεργασία, π.χ. μισθοδοσία. Στην άμεση (direct/random access), μπορούμε να προσπελάσουμε οποιαδήποτε εγγραφή κατευθείαν μέσω μιας διεύθυνσης ή ενός κλειδιού (π.χ. αρχεία σε δίσκο, βάσεις δεδομένων).

A4. Αρχείο system.ini

Το system.ini ήταν αρχείο ρυθμίσεων σε παλαιότερα Windows (3.x, 9x), που περιείχε παραμέτρους για drivers, μνήμη, γραμματοσειρές, multimedia κ.λπ. Φορτωνόταν στην εκκίνηση και επηρέαζε τον τρόπο λειτουργίας του συστήματος. Στις σύγχρονες εκδόσεις ο ρόλος του έχει αντικατασταθεί σε μεγάλο βαθμό από το Registry.

A5. FAT και NTFS

Το FAT (File Allocation Table) είναι παλαιότερο σύστημα αρχείων, απλό και υποστηριζόμενο από πολλά λειτουργικά, αλλά με περιορισμούς σε μέγεθος αρχείων, ασφάλεια και αξιοπιστία. Το NTFS (New Technology File System) είναι πιο σύγχρονο σύστημα αρχείων των Windows με υποστήριξη δικαιωμάτων πρόσβασης, κρυπτογράφησης, journaling και μεγαλύτερων δίσκων/αρχείων.

A6. Σκοπός τεχνικών διαχείρισης μνήμης

Οι τεχνικές διαχείρισης μνήμης (π.χ. συνεχής/μη συνεχής κατανομή, σελιδοποίηση, τμηματοποίηση, εικονική μνήμη) στοχεύουν στην αποδοτική χρήση της κύριας μνήμης. Επιτρέπουν την ταυτόχρονη εκτέλεση πολλών προγραμμάτων, προστατεύουν τη μνήμη κάθε διεργασίας και ελαχιστοποιούν τη σπατάλη (κατακερματισμός). Βοηθούν επίσης στην εκτέλεση προγραμμάτων μεγαλύτερων από τη φυσική μνήμη.

A7. Κατηγορίες και χαρακτηριστικά εκτυπωτών

Βασικές κατηγορίες: εκτυπωτές κρούσης (dot-matrix), inkjet, laser. Οι dot-matrix είναι φθηνοί, ανθεκτικοί, αλλά χαμηλής ποιότητας. Οι inkjet δίνουν καλή ποιότητα εικόνας/χρώματος, με μέση ταχύτητα και κόστος. Οι laser είναι υψηλής ταχύτητας και ποιότητας κειμένου, κατάλληλοι για μεγάλους όγκους εκτύπωσης.

A8. Βασικές λειτουργίες σε αρχείο

Σε ένα αρχείο μπορούμε να εκτελέσουμε: δημιουργία (create), άνοιγμα (open), ανάγνωση (read), εγγραφή/τροποποίηση (write/update), μετακίνηση δείκτη (seek), κλείσιμο (close). Επίσης διαγραφή (delete), μετονομασία (rename) και αλλαγή δικαιωμάτων πρόσβασης. Αυτές οι λειτουργίες παρέχονται από το ΛΣ μέσω system calls.

A9. Δεδομένα και πληροφορία – διαφορά

Δεδομένα είναι ακατέργαστα στοιχεία (αριθμοί, σύμβολα, μετρήσεις) χωρίς ερμηνεία από μόνα τους. Πληροφορία είναι τα δεδομένα αφού υποστούν επεξεργασία, οργάνωση και ερμηνεία, ώστε να αποκτήσουν σημασία για τον χρήστη. Η βασική διαφορά είναι ότι η πληροφορία απαντά σε ερωτήματα και υποστηρίζει λήψη αποφάσεων.

A10. Bit, byte, word

Το bit είναι η μικρότερη μονάδα πληροφορίας (0 ή 1). Το byte αποτελείται συνήθως από 8 bits και μπορεί να αναπαραστήσει έναν χαρακτήρα ή μικρούς αριθμούς. Word ονομάζεται η φυσική μονάδα δεδομένων που επεξεργάζεται η CPU (π.χ. 16, 32 ή 64 bits), ανάλογα με την αρχιτεκτονική. Τα bytes και τα words είναι σύνολα bits με διαφορετικό ρόλο.

A11. RAM και ROM – διαφορές

Η RAM (Random Access Memory) είναι πτητική μνήμη, στην οποία αποθηκεύονται προσωρινά προγράμματα και δεδομένα που εκτελούνται· χάνει το περιεχόμενο όταν κλείσει ο υπολογιστής. Η ROM (Read Only Memory) είναι μη πτητική,

προγραμματίζεται από τον κατασκευαστή ή μέσω ειδικών διαδικασιών και περιέχει συνήθως firmware (π.χ. BIOS/UEFI). Σε ROM η ανάγνωση είναι κυρίως επιτρεπτή, η εγγραφή είναι δύσκολη ή περιορισμένη.

A12. Λανθάνουσα / κρυφή μνήμη (cache)

Η cache είναι μικρή, πολύ γρήγορη μνήμη, τοποθετημένη κοντά στον επεξεργαστή, που αποθηκεύει αντίγραφα συχνά χρησιμοποιούμενων δεδομένων/εντολών από την κύρια μνήμη. Βασίζεται στην τοπικότητα αναφορών: ό,τι χρησιμοποιήθηκε πρόσφατα ή γειτονικά πιθανόν θα ξαναχρειαστεί. Έτσι μειώνει τον μέσο χρόνο πρόσβασης στη μνήμη και επιταχύνει το σύστημα.

A13. Εργασία επεξεργαστή - είδη

Ο επεξεργαστής (CPU) εκτελεί εντολές προγραμμάτων: κάνει αριθμητικές και λογικές πράξεις, ελέγχει τη ροή εκτέλεσης και συντονίζει τη μεταφορά δεδομένων μεταξύ μνήμης και συσκευών I/O. Ενδεικτικοί τύποι επεξεργαστών: Intel Core i5/i7/i9, AMD Ryzen, ARM Cortex, επεξεργαστές RISC κ.λπ. Η κύρια «εργασία» του είναι ο κύκλος ανάκλησης-αποκωδικοποίησης-εκτέλεσης εντολών.

A14. Ιός υπολογιστή - παράδειγμα

Ιός υπολογιστή είναι κακόβουλο λογισμικό που προσκολλάται σε αρχεία ή προγράμματα, αναπαράγεται και εκτελεί ανεπιθύμητες ενέργειες (καταστροφή δεδομένων, αλλοίωση συστήματος, υποκλοπή). Συνήθως χρειάζεται ενέργεια του χρήστη (άνοιγμα μολυσμένου αρχείου) για να ενεργοποιηθεί. Παράδειγμα: ιός macro σε έγγραφα Office που εκτελεί scripts κατά το άνοιγμα του εγγράφου.

A15. Επικοινωνία δεδομένων

Επικοινωνία δεδομένων είναι η μετάδοση ψηφιακών δεδομένων μεταξύ δύο ή περισσότερων συσκευών μέσω κάποιου μέσου (καλώδιο, οπτική ίνα, ασύρματο). Περιλαμβάνει τον αποστολέα, τον δέκτη, το μέσο μετάδοσης, το μήνυμα και τα πρωτόκολλα επικοινωνίας. Στόχος είναι η αξιόπιστη και έγκαιρη ανταλλαγή πληροφοριών.

A16. Κύρια στοιχεία μετάδοσης δεδομένων

Βασικά στοιχεία: πομπός (sender), δέκτης (receiver), μήνυμα (data), μέσο μετάδοσης (κανάλι: καλώδιο, οπτική ίνα, ασύρματο), πρωτόκολλο επικοινωνίας και μηχανισμοί κωδικοποίησης/συγχρονισμού. Χωρίς συμφωνημένα πρωτόκολλα, τα δεδομένα δεν θα ερμηνεύονταν σωστά από τον δέκτη.

A17. Τύποι συνεστραμμένων ζευγών

Γνωστοί τύποι: UTP (Unshielded Twisted Pair), STP (Shielded Twisted Pair), FTP (Foiled Twisted Pair). Στο UTP δεν υπάρχει θωράκιση, είναι φθηνό αλλά πιο ευαίσθητο σε παρεμβολές. Το STP/FTP έχει μεταλλική θωράκιση για μείωση θορύβου, είναι ακριβότερο και λίγο πιο δύσκολο στην εγκατάσταση.

A18. Βασικές αιτίες θορύβου

Θόρυβος είναι κάθε ανεπιθύμητη μεταβολή του σήματος κατά τη μετάδοση. Βασικές αιτίες: θερμικός θόρυβος, ηλεκτρομαγνητικές παρεμβολές (EMI), παρεμβολές από άλλα ζεύγη (crosstalk), εξασθένηση (attenuation) και παραμορφώσεις λόγω χαρακτηριστικών γραμμής. Ο θόρυβος προκαλεί σφάλματα στα λαμβανόμενα δεδομένα.

A19. Αναλογικά σήματα - χαρακτηριστικά

Αναλογικό σήμα μεταβάλλεται συνεχώς με το χρόνο και μπορεί να πάρει άπειρες ενδιάμεσες τιμές σε ένα εύρος. Χαρακτηρίζεται από πλάτος (amplitude), συχνότητα, φάση και μορφή κύματος (συνήθως ημιτονοειδής). Τυπικά παραδείγματα: φωνή, μουσική, τάση σε μικρόφωνο.

A20. Ψηφιακά σήματα - χαρακτηριστικά

Το ψηφιακό σήμα παίρνει διακριτές τιμές, συνήθως δύο (0 και 1). Εκφράζεται ως ακολουθία παλμών/τετραγωνικών κυμάτων και είναι πιο ανθεκτικό σε θόρυβο, γιατί αρκεί να ξεχωρίζει το «χαμηλό» από το «υψηλό» επίπεδο. Επιτρέπει εύκολη κωδικοποίηση, αποθήκευση και κρυπτογράφηση δεδομένων.

A21. Πότε χρειάζεται μετατροπή ψηφιακού-αναλογικού

Χρειάζεται μετατροπή όταν μια ψηφιακή συσκευή επικοινωνεί μέσω αναλογικού μέσου ή με αναλογικές συσκευές. Από ψηφιακό σε αναλογικό (DAC) π.χ. σε κάρτες ήχου για αναπαραγωγή ήχου. Από αναλογικό σε ψηφιακό (ADC) π.χ. στη δειγματοληψία ήχου/εικόνας για αποθήκευση σε ψηφιακή μορφή.

A22. Σύγχρονη vs ασύγχρονη μετάδοση

Στην ασύγχρονη μετάδοση τα δεδομένα στέλνονται χαρακτηρη-
χαρακτήρα, με bits έναρξης/λήξης (start/stop) για συγχρονισμό κάθε χαρακτήρα. Είναι απλή και οικονομική, αλλά έχει επιβάρυνση (overhead) και χαμηλότερη αποδοτικότητα. Στη σύγχρονη, τα δεδομένα στέλνονται σε blocks/frames, τα ρολόγια πομπού-δέκτη είναι συγχρονισμένα και ο ρυθμός είναι πιο αποδοτικός, κατάλληλος για μεγάλους όγκους δεδομένων.

A23. Τι εννοούμε με τον όρο «πρόβλημα»

Πρόβλημα είναι μια κατάσταση ή ανάγκη για την οποία αναζητούμε λύση· στη πληροφορική είναι μια καλά ορισμένη εργασία με δεδομένα εισόδου και ζητούμενα αποτελέσματα. Η επίλυση προβλήματος περιλαμβάνει κατανόηση, ανάλυση, σχεδίαση αλγορίθμου και υλοποίηση σε πρόγραμμα.

A24. Αλγόριθμος και πρόγραμμα

Αλγόριθμος είναι μια πεπερασμένη ακολουθία σαφών και εκτελέσιμων βημάτων που λύνει ένα πρόβλημα. Πρόγραμμα είναι η υλοποίηση ενός αλγορίθμου σε συγκεκριμένη γλώσσα προγραμματισμού, ώστε να εκτελείται από υπολογιστή. Ο ίδιος αλγόριθμος μπορεί να γραφτεί σε πολλά διαφορετικά προγράμματα.

A25. Δομή δεδομένων

Δομή δεδομένων είναι ένας οργανωμένος τρόπος αποθήκευσης και οργάνωσης δεδομένων στη μνήμη, μαζί με τις επιτρεπτές λειτουργίες πάνω σε αυτά. Παραδείγματα: πίνακες, λίστες, στοίβες, ουρές, δένδρα, γράφοι. Η επιλογή δομής επηρεάζει σημαντικά την αποδοτικότητα του αλγορίθμου.

A26. Κατηγορίες προβλημάτων ως προς τη λύση

Με κριτήριο το είδος επίλυσης, τα προβλήματα διακρίνονται σε: υπολογιστικά (απαιτούν αριθμητικούς υπολογισμούς), λογικά/αποφασιστικά (ναι/όχι), βελτιστοποίησης (εύρεση βέλτιστης λύσης) κ.ά. Επίσης σε αυτά που επιλύονται με ακριβείς αλγορίθμους και σε εκείνα που απαιτούν προσεγγιστικές/ευρετικές μεθόδους.

A27. Βασικές λειτουργίες επί δομών δεδομένων

Βασικές λειτουργίες: προσπέλαση (access) σε στοιχείο, εισαγωγή (insertion) νέου στοιχείου, διαγραφή (deletion) στοιχείου, αναζήτηση (searching) και ταξινόμηση (sorting). Επιπλέον, σε πολλές δομές υπάρχουν αντιγραφή, συγχώνευση και διαχωρισμός.

A28. Ανάλυση προβλήματος

Ανάλυση προβλήματος είναι η διαδικασία κατά την οποία διευκρινίζουμε στόχους, περιορισμούς, δεδομένα εισόδου, ζητούμενα αποτελέσματα και συνθήκες λειτουργίας. Αποσκοπεί στο να σπάσει το πρόβλημα σε μικρότερα υποπροβλήματα και να εντοπιστούν οι απαραίτητες δομές δεδομένων και διαδικασίες. Είναι κρίσιμη πριν τη σχεδίαση του αλγορίθμου.

A29. Συνηθισμένες τεχνικές σχεδίασης αλγορίθμων

Βασικές τεχνικές: τμηματικός/ιεραρχικός σχεδιασμός (top-down), διαίρει και βασίλευε (divide and conquer), δυναμικός προγραμματισμός, άπληστοι (greedy) αλγόριθμοι, backtracking. Στην εκπαίδευση χρησιμοποιούνται συχνά και εργαλεία όπως ψευδοκώδικας, διαγράμματα ροής και δομημένος προγραμματισμός (ακολουθία-επιλογή-επανάληψη).

A30. Χαρακτηριστικά αλγορίθμου

Ένας αλγόριθμος πρέπει να έχει: περατότητα (να τελειώνει σε πεπερασμένο αριθμό βημάτων), καθοριστικότητα (κάθε βήμα να είναι σαφώς ορισμένο), είσοδο, έξοδο και αποτελεσματικότητα (κάθε βήμα απλό και εκτελέσιμο). Επίσης πρέπει να είναι σωστός (να δίνει σωστά αποτελέσματα) και αποδοτικός.

A31. Τρόποι περιγραφής αλγορίθμου

Κύριοι τρόποι: φυσική γλώσσα/περιγραφή, ψευδοκώδικας, διαγράμματα ροής (flowcharts), διαγραμματικές τεχνικές (NS διαγράμματα) και τελικά κώδικας σε γλώσσα προγραμματισμού. Στην εκπαιδευτική πράξη προτιμάται ψευδοκώδικας και διαγράμματα ροής για σαφήνεια και ανεξαρτησία από γλώσσες.

A32. Πότε ένας αλγόριθμος είναι «πλήρης»

Ο αλγόριθμος θεωρείται πλήρης όταν είναι σωστός για όλες τις επιτρεπτές εισόδους, καλύπτει όλες τις πιθανές περιπτώσεις, είναι πεπερασμένος και παράγει ζητούμενη έξοδο. Πρακτικά, δεν πρέπει να αφήνει «κενά» για κάποια είσοδο ούτε να εγκλωβίζεται σε άπειρη επανάληψη.

A33. Πόσα είναι τα είδη δομής επιλογής

Στον δομημένο προγραμματισμό συνήθως αναφέρονται δύο βασικά είδη: απλή επιλογή (if...then) και σύνθετη/διπλή (if...then...else), ενώ πολλές γλώσσες υποστηρίζουν και πολλαπλή επιλογή (switch-case). Στα σχολικά βιβλία συχνά θεωρούνται τρία είδη: απλή, σύνθετη και πολλαπλή.

A34. Πόσα είναι τα είδη δομής επανάληψης

Τυπικά τρία είδη: επανάληψη με προέλεγχο (while), επανάληψη με μετρητή/καθορισμένο πλήθος (for) και επανάληψη με μετέλεγχο (do...while). Όλες υλοποιούν κυκλική εκτέλεση εντολών, αλλά διαφέρουν στο πότε ελέγχεται η συνθήκη και πώς προσδιορίζεται ο αριθμός επαναλήψεων.

A35. Καταχωρητές σε ΚΜΕ

Οι καταχωρητές είναι πολύ γρήγορες, μικρές θέσεις μνήμης εντός της CPU, που κρατούν δεδομένα, διευθύνσεις και ενδιάμεσα αποτελέσματα κατά την εκτέλεση εντολών. Παραδείγματα: Accumulator, Program Counter, Instruction

Register, MAR, MDR κ.λπ. Λόγω ταχύτητας, η χρήση τους είναι κρίσιμη για την απόδοση της ΚΜΕ.

A36. Διαφορές interpreter και compiler

Ο compiler μεταφράζει ολόκληρο το πρόγραμμα από γλώσσα υψηλού επιπέδου σε εκτελέσιμο κώδικα μηχανής πριν την εκτέλεση. Ο interpreter μεταφράζει και εκτελεί εντολή-εντολή κάθε φορά (χωρίς ενδιάμεσο εκτελέσιμο αρχείο). Με compiler η εκτέλεση είναι συνήθως πιο γρήγορη, με interpreter ευκολότερη δοκιμή/διόρθωση.

A37. Ρόλος μονάδας ελέγχου σε ΚΜΕ

Η μονάδα ελέγχου (Control Unit) «καθοδηγεί» την ΚΜΕ: ανακτά εντολές από τη μνήμη, τις αποκωδικοποιεί και παράγει σήματα ελέγχου προς τα υπόλοιπα μέρη (ALU, καταχωρητές, μνήμη, I/O). Ελέγχει τη σειρά εκτέλεσης εντολών, τα άλματα και τη διαχείριση διακοπών.

A38. Διακοπή (interrupt) και χειρισμός

Διακοπή είναι ένα ασύγχρονο σήμα προς την CPU που ζητά προσωρινή διακοπή της τρέχουσας εκτέλεσης για εξυπηρέτηση κάποιου συμβάντος (π.χ. I/O, χρονιστής, σφάλμα). Η ΚΜΕ αποθηκεύει την τρέχουσα κατάσταση (διεύθυνση επιστροφής, καταχωρητές), μεταβαίνει στη ρουτίνα εξυπηρέτησης διακοπής (ISR) και μετά επιστρέφει στο πρόγραμμα.

A39. Γιατί περισσότερη κεντρική μνήμη επιταχύνει το σύστημα
Όσο μεγαλύτερη η κύρια μνήμη, τόσο περισσότερα προγράμματα και δεδομένα χωρούν χωρίς χρήση αργής δευτερεύουσας μνήμης (δίσκου). Έτσι μειώνονται οι προσπελάσεις σε εικονική μνήμη/σελίδωση (page faults) και αυξάνεται η ταχύτητα. Ουσιαστικά περιορίζεται ο «στενός λαιμός» της αργής πρόσβασης στον δίσκο.

A40. Παγκόσμιος Ιστός (WWW) και υπηρεσίες

Ο Παγκόσμιος Ιστός (World Wide Web) είναι υπηρεσία του Internet που οργανώνει έγγραφα (ιστοσελίδες) σε μορφή υπερκείμενου με συνδέσμους. Βασίζεται κυρίως στα πρωτόκολλα HTTP/HTTPS και στη γλώσσα HTML. Προσφέρει υπηρεσίες όπως προβολή ιστοσελίδων, ηλεκτρονικά καταστήματα, web εφαρμογές, πολυμέσα κ.ά.

A41. Παγκόσμιος Ιστός - υπερκείμενο - διαφορά από απλό κείμενο

Ο Παγκόσμιος Ιστός βασίζεται στο υπερκείμενο: κείμενο με ενσωματωμένους υπερσυνδέσμους που επιτρέπουν μη γραμμική πλοήγηση. Σε αντίθεση με το απλό κείμενο που διαβάζεται σειριακά, το υπερκείμενο επιτρέπει στον χρήστη να ακολουθεί συνδέσμους προς σχετικές έννοιες, έγγραφα, πολυμέσα. Έτσι ο χρήστης καθορίζει την πορεία ανάγνωσης.

A42. Δυνατότητες υπερμεσικής δομής

Η υπερμεσική δομή επεκτείνει το υπερκείμενο σε πολλά είδη μέσων (κείμενο, εικόνα, ήχο, βίντεο, γραφικά). Προσφέρει μη γραμμική πλοήγηση μέσω υπερσυνδέσμων μεταξύ πολυμεσικού περιεχομένου, δημιουργώντας δίκτυο κόμβων-συνδέσμων. Επιτρέπει προσαρμοστική πλοήγηση, εκπαιδευτικές εφαρμογές, παρουσιάσεις και πολυμεσικά εγχειρίδια.

A43. Τεχνική bootstrap σε ανάπτυξη ιστοσελίδων

Στο web design, Bootstrap είναι πλαίσιο (framework) HTML/CSS/JavaScript που παρέχει έτοιμα components (grid, κουμπιά, φόρμες) και responsive σχεδίαση. Βοηθά στην ταχεία ανάπτυξη σελίδων που προσαρμόζονται σε διαφορετικές οθόνες (κινητά, tablets, desktop) με κοινό σύστημα πλέγματος. Μειώνει την ανάγκη για κώδικα από την αρχή και επιβάλλει συνεπή εμφάνιση.

A44. Τρόποι σύνδεσης CSS με HTML

Τρεις βασικοί τρόποι:

Inline: με το attribute style σε HTML στοιχείο.

Internal: εντολές CSS μέσα σε <style> στο <head> του εγγράφου.

External: ξεχωριστό αρχείο .css συνδεδεμένο με <link rel="stylesheet" href="...">. Ο εξωτερικός τρόπος είναι προτιμητέος για επαναχρησιμοποίηση και συντήρηση.

A45. «Ερωτήματα» (queries) στο σχεσιακό μοντέλο

Ερώτημα είναι μια έκφραση (συνήθως σε SQL) με την οποία ο χρήστης ζητά από τη βάση δεδομένων συγκεκριμένα δεδομένα σύμφωνα με κριτήρια. Μπορεί να περιλαμβάνει επιλογή στηλών, φιλτράρισμα γραμμών, ταξινόμηση, ομαδοποιήσεις και συνενώσεις πινάκων. Με τα queries αντλούμε πληροφορία από τα αποθηκευμένα δεδομένα.

A46. Σύστημα διαχείρισης βάσης δεδομένων (ΣΔΒΔ)

ΣΔΒΔ είναι το λογισμικό που επιτρέπει τον ορισμό, δημιουργία, διαχείριση και πρόσβαση σε βάσεις δεδομένων.

Παρέχει μηχανισμούς για αποθήκευση, αναζήτηση, ενημέρωση δεδομένων, ασφάλεια, ακεραιότητα, ταυτόχρονη πρόσβαση πολλών χρηστών και ανάκτηση μετά από βλάβη.

A47. Πώς απεικονίζουμε τον πραγματικό κόσμο σε μοντέλο
Χρησιμοποιούμε κυρίως το Μοντέλο Οντοτήτων-Συσχετίσεων (ER) για να περιγράψουμε οντότητες, ιδιότητες και σχέσεις του πραγματικού κόσμου. Αυτό απεικονίζεται σε διαγράμματα ER και στη συνέχεια μετατρέπεται σε σχεσιακό σχήμα (πίνακες, κλειδιά). Έτσι η εννοιολογική περιγραφή μεταφράζεται εύκολα σε ΣΔΒΔ.

A48. Σχέση «ένα προς πολλά»
Σχέση 1-N σημαίνει ότι μία οντότητα A συνδέεται με πολλές οντότητες B, αλλά κάθε B αντιστοιχεί μόνο σε μία A. Παράδειγμα: ένας Διευθυντής (manager) επιβλέπει πολλά Τμήματα, αλλά κάθε Τμήμα έχει έναν μόνο Διευθυντή. Σε σχεσιακό μοντέλο, το κλειδί της «1» πλευράς μπαίνει ως ξένο κλειδί στην «πολλά» πλευρά.

A49. Στάδια ανάπτυξης βάσης δεδομένων
Τυπικά στάδια:

Εννοιολογικός σχεδιασμός (ER διάγραμμα).

Λογικός σχεδιασμός (μετατροπή σε σχεσιακό σχήμα, κανονικοποίηση).

Φυσικός σχεδιασμός (οργάνωση σε δίσκο, ευρετήρια).

Υλοποίηση στο ΣΔΒΔ και φόρτωση δεδομένων. Ακολουθεί δοκιμή, βελτιστοποίηση και συντήρηση.

A50. Περιβάλλον πολλαπλών χρηστών
Ένα σύστημα βάσεων δεδομένων λειτουργεί σε περιβάλλον πολλαπλών χρηστών όταν επιτρέπει σε περισσότερους από έναν χρήστες/εφαρμογές να προσπελαίνουν και να τροποποιούν τα δεδομένα ταυτόχρονα. Το ΣΔΒΔ φροντίζει για μηχανισμούς συγχρονισμού, ελέγχου ταυτόχρονης πρόσβασης και αποφυγής συγκρούσεων (locks, transactions).

A51. Μοντέλα βάσεων δεδομένων και διαφορές
Κύρια μοντέλα: ιεραρχικό, δικτυωτό, σχεσιακό, αντικειμενοστραφές, αντικειμενοσχεσιακό.

Ιεραρχικό: δεδομένα σε δενδρική δομή γονέα-παιδιού.

Δικτυωτό: πιο γενικό γράφημα σχέσεων.

Σχεσιακό: δεδομένα σε πίνακες (relations) με γραμμές και στήλες.

Αντικειμενοστραφές: δεδομένα ως αντικείμενα με μεθόδους.

Το σχεσιακό κυριαρχεί λόγω απλότητας και ισχυρής θεωρίας.

A52. Χρωματικά μοντέλα

Βασικά μοντέλα: RGB, CMYK, HSV/HSB κ.ά.

RGB (Red, Green, Blue): πρόσθετο μοντέλο, για οθόνες/φωτεινές πηγές· το μαύρο είναι απουσία φωτός, το λευκό συνδυασμός των τριών.

CMYK (Cyan, Magenta, Yellow, black): αφαιρετικό, για εκτύπωση.

HSV/HSB: περιγράφει χρώμα με απόχρωση, κορεσμό, φωτεινότητα, πιο κοντά στην ανθρώπινη αντίληψη.

A53. Πρωτεύον, συμπληρωματικό, δευτερεύον χρώμα

Πρωτεύον χρώμα σε RGB: κόκκινο, πράσινο, μπλε. Συμπληρωματικό ενός πρωτεύοντος είναι το χρώμα που προκύπτει από συνδυασμό των άλλων δύο (π.χ. συμπληρωματικό του κόκκινου είναι το κυανό - συνδυασμός πράσινου+μπλε). Δευτερεύοντα είναι αυτά που προκύπτουν από μίξη δύο πρωτευόντων (π.χ. κίτρινο: κόκκινο+πράσινο). Χρησιμοποιούνται για αντίθεση και ισορροπία σε σχεδίαση.

A54. Χαρακτηριστικά ψηφιογραφικής εικόνας (bitmap)

Μια bitmap εικόνα αποτελείται από πλέγμα (grid) pixels, όπου κάθε pixel έχει συγκεκριμένο χρώμα. Χαρακτηρίζεται από διαστάσεις (πλάτος×ύψος σε pixels), ανάλυση (dpi), bit depth (bits ανά pixel) και μέγεθος αρχείου. Η μεγέθυνση οδηγεί σε «pixelization» (τετραγωνάκια).

A55. Μέγεθος ασυμπίεστης bitmap 100×100

Αριθμός pixels: $100 \times 100 = 10.000$.

α) True-color (24 bits = 3 bytes/pixel): $10.000 \times 3 = 30.000$ bytes.

β) Grayscale (8 bits = 1 byte/pixel): $10.000 \times 1 = 10.000$ bytes.

(Αγνοούμε headers/metadata, υπολογίζουμε μόνο δεδομένα εικόνας.)

A56. Τέσσερις μορφές αρχείων bitmap και χρήση
Ενδεικτικά:

BMP: ασυμπίεστο ή ελαφρά συμπιεσμένο, μεγάλα αρχεία, κυρίως σε Windows/ενδιάμεση επεξεργασία.

JPEG (JPG): απωλεστική συμπίεση, κατάλληλο για φωτογραφίες στο web.

PNG: χωρίς απώλειες, υποστηρίζει διαφάνεια, ιδανικό για γραφικά/λογότυπα.

GIF: 256 χρώματα, υποστηρίζει απλή κινούμενη εικόνα (animations).

A57. Χρήση μάσκας στην επεξεργασία εικόνας
Μάσκα (mask) είναι μια δευτέρα εικόνα ή layer που καθορίζει ποια τμήματα της κύριας εικόνας είναι ορατά ή επηρεάζονται από κάποια επεξεργασία. Συνήθως είναι grayscale: το λευκό δείχνει πλήρη εμφάνιση/επίδραση, το μαύρο πλήρη απόκρυψη. Χρησιμοποιείται για επιλεκτικές αλλαγές, σύνθεση εικόνων και εφέ διαφάνειας.

A58. Πλεονεκτήματα/μειονεκτήματα bitmap
Πλεονεκτήματα: ακριβής απεικόνιση φωτογραφιών/λεπτομερειών, συμβατότητα με πολλά προγράμματα, εύκολη επεξεργασία σε επίπεδο pixel. Μειονεκτήματα: μεγάλα αρχεία (ιδίως ασυμπίεστα), εξάρτηση από ανάλυση (χάνεται ποιότητα σε μεγέθυνση), όχι κλιμακούμενα χωρίς απώλειες.

A59. Μελέτη Σκοπιμότητας (Feasibility Study, FS)
Η Μελέτη Σκοπιμότητας εξετάζει αν ένα προτεινόμενο πληροφοριακό σύστημα είναι τεχνικά, οικονομικά, οργανωτικά και χρονικά εφικτό. Αξιολογεί κόστη-οφέλη, κινδύνους, διαθέσιμες τεχνολογίες και πόρους. Αποφασίζει αν αξίζει να προχωρήσει το έργο ή όχι.

A60. Ανάλυση Απαιτήσεων (Requirements Analysis, RA)
Η Ανάλυση Απαιτήσεων καταγράφει και αναλύει τις ανάγκες των χρηστών και του οργανισμού για το υπό ανάπτυξη σύστημα. Περιλαμβάνει συλλογή απαιτήσεων μέσω συνεντεύξεων, ερωτηματολογίων, παρατήρησης, μελέτης υπάρχοντων συστημάτων. Παράγει σύνολο λειτουργικών και μη λειτουργικών απαιτήσεων που θα ικανοποιήσει το σύστημα.

A61. Φάση προδιαγραφών απαιτήσεων (RS)

Η φάση Requirements Specification παράγει μια τυπική, δομημένη περιγραφή όλων των απαιτήσεων του συστήματος. Μετατρέπει τις (συχνά ασαφείς) ανάγκες χρήστη σε σαφείς, ελέγξιμες απαιτήσεις (λειτουργικές και μη λειτουργικές). Το έγγραφο προδιαγραφών αποτελεί τη «σύμβαση» ανάμεσα σε πελάτη και ομάδα ανάπτυξης.

A62. Φάση προδιαγραφών Λογικού Συστήματος (LS)

Στη φάση Logical System Specifications περιγράφεται τι θα κάνει το σύστημα ανεξάρτητα από την τεχνολογία υλοποίησης. Προκύπτουν λογικά μοντέλα δεδομένων (π.χ. ER), διαδικασιών (DFD) και κανόνων επιχειρησιακής λογικής. Είναι γέφυρα μεταξύ απαιτήσεων και φυσικού σχεδιασμού.

A63. Φάση Φυσικής Σχεδίασης (PD)

Η Physical Design εξειδικεύει πώς θα υλοποιηθεί το λογικό σύστημα σε συγκεκριμένο περιβάλλον (DBMS, λειτουργικά, δίκτυα). Καθορίζονται δομές αποθήκευσης, ευρετήρια, μορφές αρχείων, μηχανισμοί ασφάλειας και απόδοσης. Το αποτέλεσμα είναι λεπτομερές σχέδιο έτοιμο για υλοποίηση.

A64. Συνιστώσες ενός Πληροφοριακού Συστήματος Κλασικές συνιστώσες:

Άνθρωποι (χρήστες, διαχειριστές).

Δεδομένα.

Διαδικασίες.

Λογισμικό.

Υλικό (hardware).

Δίκτυα/επικοινωνίες.

Όλες συνεργάζονται για τη συλλογή, επεξεργασία και διανομή πληροφορίας.

A65. Τύποι λογισμικού

Το λογισμικό χωρίζεται κυρίως σε:

Λογισμικό συστήματος (λειτουργικά, drivers, utilities).

Λογισμικό εφαρμογών (word processors, browsers, ERP).

Λογισμικό προγραμματισμού (compilers, IDEs, debuggers).

Κάθε κατηγορία επιτελεί διαφορετικό ρόλο στο οικοσύστημα του Η/Υ.

A66. Φάσεις κύκλου ζωής ανάπτυξης Πληροφοριακών Συστημάτων Σε κλασικό μοντέλο:

Προκαταρκτική μελέτη/σκοπιμότητα.

Ανάλυση απαιτήσεων.

Σχεδίαση (λογική και φυσική).

Υλοποίηση/προγραμματισμός.

Έλεγχος/δοκιμές.

Εγκατάσταση και λειτουργία.

Συντήρηση/αναβάθμιση.

A67. Βασικές αρχές δομημένης προσέγγισης
Η δομημένη προσέγγιση βασίζεται σε: τμηματικό σχεδιασμό (διάσπαση σε υποσυστήματα), χρήση τριών βασικών δομών (ακολουθία, επιλογή, επανάληψη), διαγράμματα ροής/DFD και σαφή τεκμηρίωση. Στόχος είναι η ευκολότερη κατανόηση, συντήρηση και έλεγχος των συστημάτων.

A68. Βασικές αρχές μοντέλων κύκλου ζωής λογισμικού
Τα μοντέλα κύκλου ζωής (waterfall, incremental, spiral κ.λπ.) ορίζουν στάδια και ροή εργασιών στην ανάπτυξη λογισμικού. Κοινές αρχές: σαφής διαχωρισμός φάσεων, ελεγχόμενη μετάβαση μεταξύ φάσεων, τεκμηρίωση και έλεγχος ποιότητας σε κάθε στάδιο. Κάθε μοντέλο ισορροπεί διαφορετικά μεταξύ ευελιξίας και ελέγχου.

A69. Διαδικασία επιλογής αγοράς λογισμικού για ΠΣ
Περιλαμβάνει:

Καθορισμό αναγκών/απαιτήσεων.

Αναζήτηση διαθέσιμων λύσεων (COTS).

Σύγκριση χαρακτηριστικών, κόστους, υποστήριξης.

Δοκιμαστική χρήση (pilot).

Τελική επιλογή και σύμβαση προμήθειας.

Λαμβάνονται υπόψη και θέματα συμβατότητας, ασφάλειας, επεκτασιμότητας.

A70. Βασικές αρχές ελέγχου λογισμικού

Ο έλεγχος (testing) στοχεύει στην ανακάλυψη σφαλμάτων πριν την παράδοση. Αρχές: έλεγχος έναντι προδιαγραφών, κλιμάκωση (μονάδας, ολοκλήρωσης, συστήματος, αποδοχής), χρήση κατάλληλων test cases, επανάληψη ελέγχων μετά από διορθώσεις (regression testing). Δεν αποδεικνύει απουσία σφαλμάτων, αλλά μειώνει την πιθανότητα.

A71. Τεχνική ελέγχου White Box

Στο White Box testing ο ελεγκτής γνωρίζει την εσωτερική δομή/κώδικα και σχεδιάζει tests που καλύπτουν διαδρομές, κλάδους και συνθήκες του προγράμματος. Ελέγχεται η λογική ροή, οι βρόχοι, οι αποφάσεις και τα boundary conditions. Συμπληρώνει το Black Box, που εστιάζει μόνο σε είσοδο-έξοδο.

A72. Παράγοντες διασφάλισης ποιότητας λογισμικού

Η διασφάλιση ποιότητας (SQA) περιλαμβάνει διαδικασίες, πρότυπα και δραστηριότητες που εξασφαλίζουν ότι το λογισμικό ικανοποιεί απαιτήσεις. Παράγοντες: οργάνωση διαδικασιών, τήρηση προτύπων (π.χ. ISO), ανασκοπήσεις, εσωτερικοί έλεγχοι, metrics, τεκμηρίωση, εκπαίδευση προσωπικού. Στόχος είναι πρόληψη, όχι μόνο ανίχνευση σφαλμάτων.

A73. Διαχείριση ολικής ποιότητας - κριτήρια ποιότητας λογισμικού

Η διαχείριση ολικής ποιότητας (TQM) εφαρμόζει φιλοσοφία συνεχούς βελτίωσης σε όλο τον οργανισμό, με συμμετοχή όλων. Στο λογισμικό, κριτήρια ποιότητας: ορθότητα, αξιοπιστία, αποδοτικότητα, ευχρηστία, συντηρησιμότητα, φορητότητα, επαναχρησιμοποίηση. Στόχος είναι το «σωστό προϊόν» και ταυτόχρονα «σωστά παραγμένο».

A74. C++ - εμφάνιση ονόματός σας 100 φορές

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    for (int i = 0; i < 100; i++) {
```

```
        cout << "Το όνομά μου" << endl;
```

```

    }
    return 0;
}
A75. C++ - ανάγνωση 3 ακεραίων και μέγιστος
cpp
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    int maxVal = a;
    if (b > maxVal) maxVal = b;
    if (c > maxVal) maxVal = c;
    cout << "Μέγιστος: " << maxVal << endl;
    return 0;
}

```

A76. Κληρονομικότητα και πολυμορφισμός
Κληρονομικότητα: μηχανισμός με τον οποίο μια κλάση (υποκλάση/derived) κληρονομεί ιδιότητες και μεθόδους μιας άλλης (βασική/base), επιτρέποντας επαναχρησιμοποίηση κώδικα. Πολυμορφισμός: δυνατότητα να χρησιμοποιούμε κοινή διεπαφή για αντικείμενα διαφορετικών κλάσεων, όπου η κατάλληλη υλοποίηση επιλέγεται δυναμικά (virtual functions). Έτσι γράφουμε γενικό κώδικα που δουλεύει με πολλές εξειδικεύσεις.

A77. Κύρια χαρακτηριστικά της C++
Χαρακτηριστικά:

Αντικειμενοστραφής (κλάσεις, αντικείμενα, κληρονομικότητα, πολυμορφισμός, encapsulation).

Υποστηρίζει τόσο διαδικασιακό όσο και αντικειμενοστραφή προγραμματισμό.

Ισχυρός χειρισμός μνήμης (pointers, dynamic allocation).

Βιβλιοθήκη STL (containers, templates, αλγόριθμοι).

A78. Συναρτήσεις στον αντικειμενοστραφή προγραμματισμό – παράδειγμα μηνύματος
Στον ΟΟ προγραμματισμό, οι συναρτήσεις (μέθοδοι) ανήκουν σε κλάσεις και ορίζουν συμπεριφορά αντικειμένων. Ένα αντικείμενο καλεί μεθόδους άλλου αντικειμένου σαν «μήνυμα». Παράδειγμα C++:

```

cpp
class Engine {
public:
    void start() { /* ... */ }
};

class Car {
    Engine eng;
public:
    void drive() {
        eng.start(); // μήνυμα από Car προς Engine
    }
};

```

A79. Person/Student/PhDStudent/Prof - σωστό/λάθος
 Η ορθότητα εξαρτάται από τις σχέσεις κληρονομικότητας, αλλά τυπικά:

```
Person p1 = new Student();
```

```
Person p2 = new PhDStudent();
```

αφορούν Java-ύφος· σωστό conceptually αν Student και PhDStudent κληρονομούν από Person, αλλά σε C++ η δήλωση με new θα ήταν δείκτης (Person* p1 = new Student;).

PhDStudent phd1 = new Student(); είναι λάθος: δεν μπορείς να αναθέσεις αντικείμενο βάσης σε μεταβλητή υποκλάσης.

Prof t1 = new Person(); επίσης λάθος, αν Prof είναι υποκλάση.

Student s1 = new PhDStudent(); σωστό, αν PhDStudent είναι υποκλάση της Student, αλλά πάλι σε C++ θα μιλούσαμε για αντικείμενα/δείκτες, όχι new σε αναφορά τιμής.

(Αν χρειαστεί, μπορώ να το μεταφράσω και σε καθαρό C++ παράδειγμα.)

A80. Μέρη της Java Virtual Machine και τι τρέχει
 Η JVM αποτελείται ενδεικτικά από:

Class Loader (φόρτωση .class).

Bytecode Verifier.

Execution Engine (Interpreter + JIT compiler).

Runtime Data Areas (Heap, Stack, Method Area κ.λπ.).

Η JVM τρέχει bytecode μεθόδους που έχουν παραχθεί από το Java compiler (αρχεία .class) και διαχειρίζεται την εκτέλεσή τους ανεξάρτητα από το υποκείμενο λειτουργικό.

A81. Ποιοι είναι οι τύποι μεταβλητών στην Java;
Υπάρχουν δύο βασικές κατηγορίες τύπων δεδομένων στη Java:

Πρωτογενείς (primitive types): byte, short, int, long, float, double, char, boolean. Αυτοί αποθηκεύουν «απλές» τιμές (ακέραιοι, δεκαδικοί, χαρακτήρες, λογικές τιμές) και δεν είναι αντικείμενα.

Τύποι αναφοράς (reference types): κλάσεις (π.χ. String), πίνακες (π.χ. int[]), interfaces, enums κ.λπ., όπου η μεταβλητή κρατά αναφορά σε αντικείμενο στη μνήμη.

Αν μιλάμε για κατηγορίες μεταβλητών (ως προς το scope τους), η Java διακρίνει:

Τοπικές (local) μεταβλητές: μέσα σε μεθόδους/μπλοκ, ζουν όσο εκτελείται η μέθοδος.

Μεταβλητές στιγμιοτύπου (instance): ορίζονται σε κλάση, έξω από μεθόδους, και κάθε αντικείμενο έχει το δικό του αντίγραφο.

Στατικές (static) μεταβλητές: ορίζονται με static στην κλάση και είναι κοινές για όλα τα αντικείμενα της κλάσης.

A82. Ρόλος δικτύου στο βασικό client-server μοντέλο
Στο client-server μοντέλο, το δίκτυο είναι το μέσο μέσω του οποίου ο client στέλνει αιτήσεις (requests) και ο server απαντά με υπηρεσίες/δεδομένα (responses). Παρέχει μηχανισμούς δρομολόγησης, διευθυνσιοδότησης (IP) και αξιόπιστης μεταφοράς (TCP/UDP). Χωρίς δίκτυο, ο client-server διαχωρισμός δεν θα είχε νόημα.

A83. Εξυπηρετητής Εφαρμογών (Application Server)
Ο Application Server είναι ο ενδιαμέσος server που φιλοξενεί και εκτελεί την επιχειρησιακή λογική (business logic) των εφαρμογών. Αναλαμβάνει επεξεργασία αιτημάτων από τους clients, πρόσβαση σε βάσεις δεδομένων και συνδυασμό

υπηρεσιών. Επιτρέπει κεντρική διαχείριση, scaling και ασφάλεια της λογικής.

A84. Fat servers και fat clients

Σε fat client αρχιτεκτονική, ο client υλοποιεί μεγάλο μέρος της εφαρμογικής λογικής και ο server κυρίως διαχειρίζεται δεδομένα (π.χ. DB server). Σε fat server, το μεγαλύτερο μέρος της λογικής εκτελείται στον server και ο client είναι πιο «thin», υπεύθυνος κυρίως για παρουσίαση. Κάθε προσέγγιση έχει επιπτώσεις σε απόδοση, διαχείριση και ασφάλεια.

A85. Βήματα μιας πλήρους συναλλαγής στο client-server

Τυπικά:

Ο client ξεκινά αίτημα (π.χ. login, εκτέλεση λειτουργίας).

Το αίτημα στέλνεται μέσω δικτύου στον server.

Ο server ελέγχει δικαιώματα, εκτελεί λογική εφαρμογής, προσπελαύνει τη βάση δεδομένων.

Δημιουργεί απάντηση με αποτελέσματα ή κωδικούς σφάλματος.

Η απάντηση επιστρέφει στον client, ο οποίος ενημερώνει το χρήστη ή συνεχίζει με επόμενη ενέργεια.

A86. Τρεις αλγόριθμοι κρυπτογράφησης

Ενδεικτικό τρίο:

Συμμετρικοί: AES, 3DES.

Ασύμμετροι: RSA, ECC.

Hash (αν και δεν είναι «κρυπτογράφηση», συχνά αναφέρεται μαζί): SHA-256 κ.ά.

Κάθε ένας εξυπηρετεί διαφορετικό ρόλο (εμπιστευτικότητα, ακεραιότητα, πιστοποίηση).

A87. Κατηγορίες αλγορίθμων κρυπτογράφησης

Βασικές κατηγορίες:

Συμμετρική κρυπτογράφηση (ένα κλειδί για κρυπτογράφηση/αποκρυπτογράφηση: AES, 3DES).

Ασύμμετρη κρυπτογράφηση (ζεύγος δημοσίου-ιδιωτικού κλειδιού: RSA, ECC).

Αλγόριθμοι κατακερματισμού (hash, μονής κατεύθυνσης: SHA, MD5).

Χρησιμοποιούνται συχνά σε συνδυασμό.

A88. Τι θεωρούμε «απειλή» σε υπολογιστικό σύστημα
Απειλή (threat) είναι κάθε πιθανό συμβάν ή ενέργεια που μπορεί να προκαλέσει ζημιά σε πόρους του συστήματος (δεδομένα, υπηρεσίες, υποδομή). Παραδείγματα: κακόβουλο λογισμικό (malware), μη εξουσιοδοτημένη πρόσβαση (hacking), απώλεια δεδομένων, DoS επιθέσεις. Η απειλή αξιολογείται με βάση πιθανότητα και αντίκτυπο.

A89. Phishing και αντιμετώπιση
Phishing είναι τεχνική κοινωνικής μηχανικής, όπου ο επιτιθέμενος προσπαθεί να παραπλανήσει τον χρήστη (με ψεύτικα emails/σελίδες) ώστε να αποκαλύψει κωδικούς, στοιχεία καρτών κ.λπ. Αντιμετώπιση: προσοχή σε ύποπτα links, χρήση HTTPS, επαλήθευση αποστολέα, αντι-phishing φίλτρα, εκπαίδευση χρηστών και 2FA (two-factor authentication).

A90. Ψηφιακό πιστοποιητικό
Ψηφιακό πιστοποιητικό είναι ηλεκτρονικό έγγραφο που συνδέει την ταυτότητα ενός υποκειμένου (server, χρήστη) με το δημόσιο κλειδί του, υπογεγραμμένο από μια Αρχή Πιστοποίησης (CA). Χρησιμοποιείται σε πρωτόκολλα όπως TLS/SSL για να επιβεβαιώνεται ότι συνδεόμαστε με τον σωστό server. Περιέχει όνομα, δημόσιο κλειδί, ημερομηνίες ισχύος κ.λπ.

A91. Τρόποι ανίχνευσης απειλών σε υπολογιστικό σύστημα
Μέθοδοι:

Αντι-ιικά/antimalware προγράμματα με signature και heuristic scanning.

IDS/IPS (Intrusion Detection/Prevention Systems) που ανιχνεύουν ύποπτη κίνηση.

Παρακολούθηση log files για ανωμαλίες.

Vulnerability scanners και penetration tests.

Συμπληρώνονται από σωστές πολιτικές ενημέρωσης και patching.

A92. Οντότητα, Συσχέτιση, Βαθμός, Πολυπλοκότητα (ER μοντέλο)

Οντότητα (entity): αντικείμενο/έννοια του πραγματικού κόσμου με ανεξάρτητη ύπαρξη (π.χ. Υπάλληλος, Τμήμα).

Συσχέτιση (relationship): σύνδεση μεταξύ οντοτήτων (π.χ. Υπάλληλος-ανήκει-Τμήμα).

Βαθμός συσχέτισης (degree): πόσοι τύποι οντοτήτων συμμετέχουν (π.χ. binary, ternary).

Πολυπλοκότητα συσχέτισης (cardinality): 1-1, 1-N, N-M κ.λπ. (πόσες οντότητες της μίας σχετίζονται με πόσες της άλλης).

A93. Τρία επίπεδα αρχιτεκτονικής συστήματος βάσης δεδομένων Σύμφωνα με την ANSI/SPARC:

Εξωτερικό επίπεδο (views): πώς βλέπουν τα δεδομένα οι χρήστες.

Νοητικό επίπεδο (conceptual): συνολική λογική δομή της βάσης (έννοιες, σχέσεις).

Εσωτερικό επίπεδο (internal): φυσική αποθήκευση σε δίσκους, ευρετήρια.

Ο διαχωρισμός αυτός υποστηρίζει ανεξαρτησία δεδομένων.

A94. Τύποι σχέσεων μεταξύ δύο οντοτήτων
Κύριοι τύποι:

Ένα-προς-ένα (1-1).

Ένα-προς-πολλά (1-N).

Πολλά-προς-πολλά (N-M).

Η σχέση 1-N υλοποιείται με ξένο κλειδί στον «πολλά» πίνακα, ενώ N-M συνήθως με ενδιάμεσο πίνακα συσχέτισης.

A95. SQL - εμφάνιση DNAME και DEPTNO από DEPT
sql
SELECT DNAME, DEPTNO
FROM DEPT;

A96. SQL - στοιχεία υπαλλήλων του τμήματος 30

```
sql
SELECT *
FROM EMP
WHERE DEPTNO = 30;
```

A97. SQL - επώνυμα, κωδικοί, όνομα/κωδικός τμήματος, εργασία για γραμματείς
Υποθέτουμε ότι:

JOB = 'ΓΡΑΜΜΑΤΕΑΣ'

πίνακας DEPT με DNAME, DEPTNO.

```
sql
SELECT E.ENAME, E.EMPNO,
       D.DNAME, D.DEPTNO,
       E.JOB
FROM EMP E
JOIN DEPT D ON E.DEPTNO = D.DEPTNO
WHERE E.JOB = 'ΓΡΑΜΜΑΤΕΑΣ';
```

A98. Rollback σε βάση δεδομένων

Rollback είναι η λειτουργία με την οποία αναιρούνται όλες οι αλλαγές (INSERT/UPDATE/DELETE) μιας συναλλαγής που δεν ολοκληρώθηκε σωστά (δεν έγινε COMMIT). Επαναφέρει τα δεδομένα στην κατάσταση πριν την έναρξη της συναλλαγής, διασφαλίζοντας ατομικότητα (Atomicity) των transactions.

A99. Απειλές για ασφάλεια βάσης δεδομένων

Απειλές: μη εξουσιοδοτημένη πρόσβαση, αλλοίωση/διαγραφή δεδομένων, υποκλοπή δεδομένων (sniffing), SQL injection, φυσικές βλάβες (δίσκοι, πυρκαγιά). Αντιμετώπιση με μηχανισμούς authentication/authorization, κρυπτογράφηση, backups, firewalls και έλεγχο εφαρμογών.

A100. Χρήση log files σε βάση δεδομένων

Τα αρχεία καταγραφής επανάληψης (log files) καταγράφουν όλες τις συναλλαγές και τις αλλαγές στα δεδομένα. Χρησιμοποιούνται για ανάκτηση (recovery) μετά από βλάβη: επανάληψη (redo) committed συναλλαγών και ακύρωση (undo) μη completed. Είναι κρίσιμα για την ακεραιότητα και αξιοπιστία της βάσης.

Πάμε ακριβώς στις 101-111, όπως τις ζήτησες.

101. SQL - πρόσθεση πεδίων Address1 και Zip1 στον πίνακα EMP

Αν «πρόσθεσετε τις πλειάδες Address1 και Zip1» εννοεί νέα πεδία/στήλες στον πίνακα EMP, τότε:

```
sql
ALTER TABLE EMP
ADD Address1 VARCHAR(50),
     Zip1      VARCHAR(10);
```

102. SQL - όνομα, θέση, μισθός υπαλλήλων τμήματος 20 με SAL+COMM > 2000

```
sql
SELECT ENAME, JOB, SAL
FROM   EMP
WHERE  DEPTNO = 20
      AND (SAL + NVL(COMM, 0)) > 2000; //null value logic
Σε SQL Server/MySQL, αντί για NVL θα χρησιμοποιούσες π.χ.
ISNULL(COMM,0) ή IFNULL(COMM,0).
```

103. SQL - συνολική αμοιβή (SAL+COMM) ανά πωλητή
Υποθέτοντας ότι οι πωλητές έχουν JOB = 'ΠΩΛΗΤΗΣ':

```
sql
SELECT ENAME,
       (SAL + NVL(COMM, 0)) AS TOTAL_PAY
FROM   EMP
WHERE  JOB = 'ΠΩΛΗΤΗΣ';
```

Αν ζητά «άθροισμα ανά πωλητή» σε περίπτωση πολλών εγγραφών ανά άτομο:

```
sql
SELECT ENAME,
       SUM(SAL + NVL(COMM, 0)) AS TOTAL_PAY
FROM   EMP
WHERE  JOB = 'ΠΩΛΗΤΗΣ'
GROUP BY ENAME;
```

104. SQL - ενημέρωση μισθού υπαλλήλων τμήματος 20 με +300 €

```
sql
UPDATE EMP
SET    SAL = SAL + 300
WHERE  DEPTNO = 20;
Η εντολή ενημερώνει όλα τα rows του πίνακα EMP με
DEPTNO=20, προσθέτοντας 300 στον τρέχοντα μισθό.
```

105. SQL – ποιοι είναι οι πιο καλοπληρωμένοι υπάλληλοι ανά είδος εργασίας
Θέλουμε ανά JOB τον/τους υπαλλήλους με μέγιστο SAL+COMM:

```
sql
SELECT E.ENAME, E.JOB, E.SAL, E.COMM
FROM   EMP E
JOIN (
    SELECT JOB,
           MAX(SAL + NVL(COMM, 0)) AS MAX_PAY
    FROM   EMP
    GROUP BY JOB
) M
ON E.JOB = M.JOB
AND (E.SAL + NVL(E.COMM, 0)) = M.MAX_PAY;
```

Έτσι παίρνουμε όλους τους υπαλλήλους που ισοβαθμούν στο μέγιστο ανά είδος εργασίας.

106. SQL – επώνυμο και όνομα τμήματος υπαλλήλων-γραμματέων
Υποθέτοντας ENAME = επώνυμο και πίνακα DEPT με DNAME (όνομα τμήματος):

```
sql
SELECT E.ENAME,
       D.DNAME
FROM   EMP E
JOIN   DEPT D ON E.DEPTNO = D.DEPTNO
WHERE  E.JOB = 'ΓΡΑΜΜΑΤΕΑΣ';
```

Αν υπάρχει ξεχωριστό πεδίο για όνομα, θα προστεθεί στη λίστα SELECT.

107. Πέντε βασικοί τύποι δεδομένων
Ενδεικτικά πέντε τύποι που χρησιμοποιούνται ευρέως σε σχεσιακές βάσεις:

Ακέραιος αριθμός: INT / INTEGER.

Μικρότερος ακέραιος: SMALLINT.

Πραγματικός/δεκαδικός: FLOAT, REAL, DECIMAL(p,s).

Συμβολοσειρά σταθερού/μεταβλητού μήκους: CHAR(n), VARCHAR(n).

Ημερομηνία/ώρα: DATE, TIMESTAMP.

108. Τέσσερις βασικές πράξεις συνόλου της σχεσιακής
άλγεβρας με παράδειγμα
Ένωση (Union) -

R
U
S

$R \cup S$:

Παράδειγμα: εργαζόμενοι από δύο πίνακες EMP1 και EMP2:

R
=

R= υπάλληλοι Αθήνα,

S
=

S= υπάλληλοι Θεσ/νίκη →

R
U

S

$R \cup S$ = όλοι οι υπάλληλοι από Αθήνα ή Θεσ/νίκη.

Τομή (Intersection) -

R
 $\cap$
S

$R \cap S$:

Π.χ. υπάλληλοι που υπάρχουν και σε EMP1 και σε EMP2 (κοινό
υποσύνολο) .

Διαφορά (Difference) -

R
-
S

$R - S$:

Π.χ. υπάλληλοι σε EMP1 που δεν εμφανίζονται στον EMP2.

Καρτεσιανό γινόμενο (Cartesian Product) -

R
 $\times$
S

R×S:

Π.χ. συνδυασμός κάθε υπαλλήλου με κάθε έργο (Employee × Project), πριν εφαρμοστεί κάποια συνθήκη join.

109. Πέντε βασικές λειτουργίες/υπηρεσίες ΣΔΒΔ

Ορισμός δεδομένων (DDL): δημιουργία, τροποποίηση και διαγραφή αντικειμένων (CREATE/ALTER/DROP TABLE, INDEX, VIEW) .

Χειρισμός δεδομένων (DML): INSERT, UPDATE, DELETE, SELECT για διαχείριση εγγραφών.

Έλεγχος πρόσβασης και ασφάλειας: χρήστες, ρόλοι, δικαιώματα (GRANT/REVOKE) .

Διαχείριση συναλλαγών: COMMIT, ROLLBACK, locking, ώστε να εξασφαλίζονται ιδιότητες ACID.

Ανάκτηση και backup: μηχανισμοί backup/restore και recovery μετά από βλάβη.

110. Τα τρία επίπεδα στην αρχιτεκτονική τριών επιπέδων και οι στόχοι τους

Εσωτερικό (internal) επίπεδο: περιγράφει πώς ακριβώς αποθηκεύονται τα δεδομένα σε φυσικό επίπεδο (αρχεία, ευρετήρια, κατανομές). Στόχος: βελτιστοποίηση αποθήκευσης και απόδοσης.

Εννοιολογικό (conceptual) επίπεδο: λογική δομή της βάσης για όλο τον οργανισμό (πίνακες, σχέσεις, περιορισμοί). Στόχος: συνοχή και ανεξαρτησία από φυσικές λεπτομέρειες.

Εξωτερικό (external) επίπεδο: όψεις (views) για διαφορετικές ομάδες χρηστών. Στόχος: προσαρμοσμένη πρόσβαση, ασφάλεια και ευκολία χρήσης.

111. Ανεξαρτησία δεδομένων και δύο βασικές κατηγορίες
Ανεξαρτησία δεδομένων είναι η δυνατότητα να τροποποιείται το σχήμα/οργάνωση της βάσης σε ένα επίπεδο χωρίς να απαιτούνται αλλαγές στο αμέσως ανώτερο επίπεδο ή στις εφαρμογές.

Φυσική ανεξαρτησία: αλλαγές στην εσωτερική/φυσική οργάνωση (δομές αρχείων, ευρετήρια, τρόπος αποθήκευσης) δεν επηρεάζουν το εννοιολογικό σχήμα.

Λογική ανεξαρτησία: αλλαγές στο εννοιολογικό σχήμα (π.χ. διάσπαση/συγχώνευση πινάκων) δεν απαιτούν τροποποίηση των εξωτερικών όψεων και των προγραμμάτων που τις χρησιμοποιούν (ή απαιτούν μόνο ελάχιστες).